Comparative Performance Evaluation of NFT Marketplace Implementations on Sui, Aptos, and Solana Blockchains

Chapter 1: Introduction

1.1 Background

Blockchain technology has transformed the management of digital assets by enabling decentralized ownership, transparent transaction processing, and immutable record keeping. Among the numerous blockchain applications developed over the past decade, **Non-Fungible Tokens (NFTs)** have emerged as one of the most influential innovations, allowing the representation of unique digital assets on distributed ledger systems. Unlike cryptocurrencies, NFTs possess distinct identities and cannot be exchanged on a one-to-one basis, making them suitable for applications such as digital art, gaming assets, music, collectibles, intellectual property, virtual real estate, and identity management.

The rapid adoption of NFTs has resulted in the development of decentralized marketplaces where creators and users can mint, trade, and manage digital assets without relying on centralized intermediaries. Platforms such as OpenSea, Magic Eden, and Blur have demonstrated the commercial viability of NFT ecosystems while simultaneously highlighting the scalability challenges faced by existing blockchain infrastructures. Increasing transaction volumes often lead to network congestion, higher transaction costs, and longer confirmation times, reducing the overall usability of decentralized marketplaces.

To address these limitations, several next-generation Layer-1 blockchain platforms have introduced new execution models designed to improve scalability without compromising decentralization or security. Among the most prominent of these are **Sui**, **Aptos**, and **Solana**, each employing fundamentally different architectural approaches to transaction processing and smart contract execution.

Sui adopts an **object-centric execution model**, representing digital assets as independently owned objects that can be processed in parallel whenever transaction dependencies do not overlap. This architecture aims to maximize scalability while maintaining strict ownership semantics through the Move programming language.

Aptos also utilizes the Move programming language but introduces a **resource-oriented execution model** combined with the **Block-STM parallel execution engine**, allowing independent transactions to execute concurrently while preserving deterministic execution through optimistic concurrency control.

Solana employs a distinct architecture based on **Proof of History (PoH)**, **Proof of Stake (PoS)**, and an **account-based execution model** implemented primarily in Rust. By establishing

transaction ordering prior to consensus and enabling parallel execution across independent accounts, Solana achieves high throughput while maintaining extremely low transaction fees.

Although each blockchain claims improvements in scalability and performance, meaningful comparisons require identical decentralized applications to be implemented and evaluated under comparable conditions. Performance claims based solely on theoretical blockchain specifications or isolated benchmark studies often fail to capture the practical behaviour experienced by decentralized applications during real-world deployment.

To address this limitation, this project develops an identical NFT marketplace on Sui, Aptos, and Solana using each platform's recommended development framework and programming language. Each implementation provides equivalent functionality—including NFT minting, marketplace listing, purchasing, and delisting—allowing direct comparison of transaction latency, throughput, transaction cost, deployment complexity, security characteristics, and development experience.

The resulting comparative analysis provides practical insight into the strengths and limitations of each blockchain architecture while identifying the platform most suitable for decentralized NFT marketplace applications.

1.2 Problem Statement

The widespread adoption of decentralized applications has intensified the demand for blockchain platforms capable of supporting high transaction throughput, low latency, and economical transaction costs. While first-generation smart contract platforms successfully introduced decentralized programmability, many continue to experience scalability limitations arising from sequential transaction execution, network congestion, and increasing computational overhead. These challenges become particularly evident within NFT marketplaces, where large numbers of users simultaneously perform asset creation, trading, ownership transfer, and marketplace management operations.

Modern Layer-1 blockchains such as Sui, Aptos, and Solana have introduced innovative execution models intended to overcome these limitations. Despite their growing adoption, direct comparisons of these platforms remain limited because existing studies often evaluate different applications, employ inconsistent benchmarking methodologies, or rely primarily on theoretical performance claims rather than practical deployment.

Furthermore, differences in smart contract programming languages, execution architectures, storage models, and consensus mechanisms make objective evaluation difficult without implementing functionally equivalent decentralized applications across all platforms.

Consequently, developers and organizations seeking to build scalable NFT marketplaces face uncertainty when selecting an appropriate blockchain platform. Questions remain regarding transaction performance, deployment complexity, operational cost, scalability, and security under realistic application workloads.

This research addresses these challenges by implementing the same NFT marketplace on Sui, Aptos, and Solana, deploying each implementation on its respective public test network, and conducting performance benchmarking using identical evaluation criteria. The resulting comparison provides an evidence-based assessment of the strengths and limitations of each blockchain platform for decentralized NFT marketplace development.

1.3 Aim of the Study

The primary aim of this research is to design, implement, deploy, and comparatively evaluate functionally equivalent NFT marketplace applications on the **Sui**, **Aptos**, and **Solana** blockchains in order to assess their suitability for decentralized digital asset trading.

The study investigates how differences in blockchain architecture, execution model, programming language, and storage mechanisms influence transaction performance, operational cost, scalability, security, and overall development experience.

1.4 Objectives

The objectives of this research are:

- To study the architectural characteristics of the Sui, Aptos, and Solana blockchains.
- To design and implement equivalent NFT marketplace applications using the native smart contract framework of each blockchain.
- To deploy each implementation successfully on its respective public test network.
- To validate the functional correctness of NFT minting, listing, purchasing, and delisting operations.
- To benchmark transaction latency, throughput, and operational cost under comparable testing conditions.
- To conduct security analysis of each implementation through smart contract review and platform-specific security mechanisms.
- To compare the performance, scalability, security, and implementation complexity of the three blockchain platforms.

- To identify deployment challenges encountered during migration between blockchain ecosystems.
- To propose optimization strategies and future improvements for cross-chain NFT marketplace development.

1.5 Research Questions

This study seeks to answer the following research questions:

RQ1: Which blockchain platform demonstrates the lowest transaction latency for NFT marketplace operations?

RQ2: How does transaction throughput compare across Sui, Aptos, and Solana when executing identical marketplace operations?

RQ3: Which blockchain provides the lowest operational cost for NFT marketplace transactions?

RQ4: How do the architectural differences of Sui, Aptos, and Solana influence transaction performance and scalability?

RQ5: Which blockchain offers the strongest security mechanisms for decentralized NFT marketplace development?

RQ6: What implementation and deployment challenges arise when developing equivalent applications across multiple blockchain ecosystems?

RQ7: Which blockchain platform provides the most balanced combination of performance, security, scalability, and developer experience for NFT marketplace applications?

1.6 Scope of the Study

This research focuses on the comparative implementation and evaluation of NFT marketplace applications developed on three modern Layer-1 blockchain platforms: Sui, Aptos, and Solana. Each implementation provides identical core marketplace functionality, including NFT minting, listing, purchasing, and delisting, enabling direct comparison of performance and architectural characteristics.

The evaluation includes smart contract implementation, deployment on public test networks, functional validation, security analysis, transaction latency measurement, throughput evaluation, and transaction cost analysis. Benchmarking is conducted using command-line tools and custom scripts under comparable testing conditions to ensure consistency across platforms.

The study does not evaluate mainnet performance, large-scale distributed workloads, validator-level optimizations, or economic aspects such as token valuation. Similarly, interoperability mechanisms, cross-chain bridges, and advanced marketplace features—including auctions, royalties, and decentralized governance—are identified as future enhancements rather than components of the current implementation.

1.7 Significance of the Study

The findings of this research contribute to both academic research and practical blockchain application development. From an academic perspective, the study provides a systematic comparison of three emerging blockchain platforms using identical decentralized applications and a consistent benchmarking methodology. This approach enables objective evaluation of blockchain architectures while reducing inconsistencies commonly found in previous comparative studies.

From an industrial perspective, the results provide developers and organizations with practical guidance when selecting blockchain platforms for NFT marketplace development. By comparing transaction latency, throughput, transaction cost, implementation complexity, and security characteristics, the study identifies the strengths and limitations of each platform under realistic deployment conditions.

Furthermore, the migration challenges, optimization strategies, and improvement roadmap presented in this report provide valuable insights for future cross-chain decentralized application development and contribute to the broader understanding of scalable blockchain architectures.

Chapter 2: Literature Review

2.1 Introduction

Blockchain technology has evolved significantly since the introduction of Bitcoin, expanding beyond digital currencies to support decentralized applications (dApps), decentralized finance (DeFi), supply chain management, identity systems, healthcare, and digital asset ownership. Among these applications, **Non-Fungible Tokens (NFTs)** have become one of the fastest-growing sectors, enabling the representation and exchange of unique digital assets on decentralized networks.

The increasing popularity of NFTs has stimulated the development of decentralized marketplaces where creators and users can securely mint, trade, and manage digital assets without centralized intermediaries. However, the rapid growth of blockchain applications has also exposed limitations in many existing blockchain platforms, particularly regarding scalability, transaction throughput, confirmation latency, and transaction costs.

To overcome these challenges, several next-generation blockchain platforms—including **Sui**, **Aptos**, and **Solana**—have introduced innovative execution models and consensus mechanisms designed to improve blockchain performance while maintaining security and decentralization.

This chapter reviews existing research related to blockchain scalability, NFT marketplaces, smart contract platforms, performance benchmarking, and blockchain security. It also identifies the research gap addressed by this project.

2.2 Evolution of Blockchain Technology

Blockchain technology was initially introduced through Bitcoin in 2008 as a decentralized peer-to-peer electronic cash system. Bitcoin demonstrated that distributed consensus could eliminate the need for centralized financial institutions while maintaining transaction integrity through cryptographic verification.

Although Bitcoin successfully solved the problem of decentralized digital payments, its scripting language provided only limited programmability. This limitation motivated the development of **Ethereum**, which introduced programmable smart contracts capable of executing arbitrary business logic directly on the blockchain. Ethereum significantly expanded blockchain applications by enabling decentralized finance, decentralized autonomous organizations, gaming platforms, and NFT ecosystems.

Despite its flexibility, Ethereum experiences scalability challenges because transactions are processed sequentially by every validator. During periods of high network demand, congestion results in increased confirmation times and higher transaction fees, making certain applications economically impractical.

To address these limitations, newer Layer-1 blockchain platforms have adopted fundamentally different execution models. Rather than relying solely on sequential execution, modern blockchains increasingly employ parallel transaction processing, optimized consensus algorithms, and efficient state management to improve scalability.

Examples include Sui's object-centric architecture, Aptos' Block-STM parallel execution engine, and Solana's Proof of History consensus mechanism. These innovations form the technological foundation for high-performance decentralized applications such as NFT marketplaces.

2.3 Non-Fungible Tokens (NFTs)

Non-Fungible Tokens represent unique blockchain-based assets that differ fundamentally from cryptocurrencies. While cryptocurrencies such as Bitcoin and Ethereum are fungible and interchangeable, each NFT possesses a distinct identity and unique metadata that cannot be replaced by another token.

NFTs are commonly used to represent:

- Digital artwork
- Gaming assets
- Music and multimedia
- Virtual real estate
- Collectibles
- Academic certificates
- Identity credentials
- Intellectual property

Ownership of NFTs is permanently recorded on the blockchain, providing transparent provenance and preventing unauthorized duplication.

The widespread adoption of NFTs has transformed digital ownership by enabling creators to monetize digital content directly while allowing buyers to verify authenticity through decentralized records.

As NFT ecosystems continue to expand, blockchain platforms must efficiently support large numbers of asset creation, ownership transfers, and marketplace transactions without sacrificing security or scalability.

2.4 NFT Marketplaces

NFT marketplaces provide decentralized platforms where users can create, list, purchase, and manage digital assets.

Typical marketplace functionality includes:

- NFT minting
- Marketplace listing
- Purchasing
- Delisting
- Ownership transfer
- Creator royalties
- Marketplace fees

Popular NFT marketplaces include OpenSea, Magic Eden, Blur, Rarible, and Tensor. These platforms demonstrate the commercial viability of NFT ecosystems while also highlighting the technical challenges associated with supporting millions of users and transactions.

Centralized marketplaces often provide superior user experience but require users to trust platform operators. In contrast, decentralized NFT marketplaces execute transactions entirely through smart contracts, providing greater transparency and eliminating dependence on centralized intermediaries.

However, decentralized marketplaces also inherit the performance limitations of the underlying blockchain platform. Consequently, selecting an appropriate blockchain architecture becomes a critical design decision for NFT marketplace development.

2.5 Blockchain Scalability

Scalability remains one of the most significant challenges facing blockchain technology. An ideal blockchain should simultaneously provide:

- High transaction throughput
- Low confirmation latency
- Low transaction costs
- Strong security
- Decentralization

Achieving all of these objectives simultaneously is difficult because improvements in one characteristic often affect others, a challenge commonly referred to as the **Blockchain Trilemma**.

Traditional blockchain platforms typically execute transactions sequentially, limiting throughput and increasing confirmation delays during periods of high demand. As blockchain adoption grows, scalable execution mechanisms become increasingly important for supporting decentralized applications.

Modern blockchain platforms employ various approaches to improve scalability, including:

- Parallel transaction execution
- Optimized consensus protocols
- Object-oriented storage
- Resource-oriented programming
- Account-based execution
- Sharding
- Layer-2 scaling solutions

Each blockchain platform adopts a unique combination of these techniques depending on its architectural objectives.

2.6 Sui Blockchain

Sui is a modern Layer-1 blockchain developed using the Move programming language. Unlike conventional account-based blockchains, Sui organizes blockchain state as independently owned objects.

This object-centric architecture enables independent transactions involving different objects to execute in parallel without requiring global consensus. Consequently, Sui can significantly reduce transaction latency while improving scalability for decentralized applications involving independently owned assets such as NFTs.

Move's ownership model further improves security by preventing accidental duplication or unauthorized modification of digital assets.

These characteristics make Sui particularly attractive for NFT marketplaces where ownership integrity and independent asset management are fundamental requirements.

2.7 Aptos Blockchain

Aptos is another Layer-1 blockchain built using the Move programming language but adopts a resource-oriented programming model rather than an object-oriented storage model.

Digital assets are represented as Move resources stored within user accounts. Resources cannot be copied, implicitly destroyed, or accidentally duplicated, providing strong ownership guarantees.

One of Aptos' most significant innovations is the **Block-STM** execution engine, which enables speculative parallel execution of independent transactions while automatically resolving conflicts.

This architecture combines security with improved scalability, making Aptos suitable for decentralized applications requiring frequent asset transfers and high transaction throughput.

2.8 Solana Blockchain

Solana is a high-performance blockchain that combines **Proof of History** with **Proof of Stake** to achieve efficient transaction processing.

Unlike Move-based blockchains, Solana employs an account-oriented architecture implemented primarily using the Rust programming language.

Transactions specify every account involved during execution, enabling Solana's runtime to execute independent transactions concurrently whenever account dependencies permit.

Additionally, Solana maintains extremely low transaction fees, making it particularly attractive for applications requiring frequent user interactions such as gaming platforms and NFT marketplaces.

The Anchor Framework has further simplified Solana smart contract development by providing automatic account validation and structured program development.

2.9 Smart Contract Security

Smart contract vulnerabilities remain one of the greatest risks associated with decentralized applications.

Common vulnerabilities include:

- Reentrancy attacks
- Integer overflow

- Unauthorized access
- Asset duplication
- Improper ownership validation
- State inconsistency

Move-based blockchains such as Sui and Aptos address many of these risks through resource-oriented programming, strict ownership semantics, and compile-time verification.

Solana emphasizes secure account validation, signer verification, and Program Derived Addresses to prevent unauthorized account manipulation.

Formal verification tools such as **Move Prover** further improve smart contract reliability by mathematically verifying contract correctness before deployment.

2.10 Performance Benchmarking in Previous Studies

Numerous studies have evaluated blockchain performance using metrics such as transaction throughput, confirmation latency, gas consumption, and execution cost. However, these studies often focus on individual blockchain platforms or use different applications and benchmarking methodologies, making direct comparison difficult.

Research on Ethereum frequently highlights scalability limitations caused by sequential execution and increasing gas costs during network congestion. Studies on Solana generally emphasize its high throughput and low transaction fees, while investigations into Move-based blockchains focus on the security and scalability benefits of object-oriented and resource-oriented programming models.

Despite these contributions, relatively few studies have implemented the **same decentralized application** across multiple modern Layer-1 blockchains and evaluated them under comparable experimental conditions. This lack of consistent benchmarking makes it challenging to determine how architectural differences influence the practical performance of decentralized applications such as NFT marketplaces.

2.11 Research Gap

The literature indicates substantial progress in blockchain scalability, smart contract security, and decentralized application development. However, several important gaps remain.

Most existing benchmarking studies evaluate different applications across different blockchain platforms, making objective comparison difficult. Others rely primarily on

theoretical throughput claims or synthetic benchmarks rather than real-world decentralized applications deployed on public test networks.

Furthermore, comparative studies involving **Sui**, **Aptos**, and **Solana** remain limited, particularly in the context of NFT marketplace development. The architectural differences among these platforms—including object-centric execution, resource-oriented programming, and account-based execution—have not been extensively evaluated using identical smart contract functionality and consistent benchmarking methodologies.

This project addresses these limitations by designing, implementing, and deploying functionally equivalent NFT marketplaces on all three blockchain platforms. Each implementation is evaluated using identical performance metrics, including transaction latency, throughput, operational cost, deployment complexity, and security considerations. By adopting a consistent experimental methodology, the study provides an objective comparison of three leading Layer-1 blockchain platforms for decentralized NFT marketplace applications.

Chapter 3: System Design and Architecture

3.1 Introduction

To ensure an objective comparison between blockchain platforms, a functionally equivalent NFT marketplace was developed on Sui, Aptos, and Solana. Although each blockchain employs a distinct execution model, storage mechanism, and smart contract language, the marketplace was designed to provide identical core functionality across all three implementations. This approach ensures that performance differences arise primarily from blockchain architecture rather than application logic.

The marketplace enables users to create digital assets, offer them for sale, purchase listed assets, and remove listings. Each implementation follows the same operational workflow while adapting to the architectural characteristics of its respective blockchain.

This chapter presents the overall system architecture, describes the common marketplace workflow, explains the functional components, and discusses how the marketplace design was adapted for each blockchain platform.

3.2 Overall System Architecture

The NFT marketplace follows a modular client–blockchain architecture consisting of four primary layers:

- **User Layer**
- **Wallet Layer**
- **Blockchain Layer**
- **Smart Contract Layer**

Users interact with the marketplace through blockchain wallets that authenticate transactions and digitally sign marketplace operations. Signed transactions are submitted to the blockchain network, where smart contracts validate requests, update marketplace state, and record ownership changes on-chain.

The marketplace supports the complete lifecycle of NFT ownership, beginning with asset creation and continuing through listing, purchasing, ownership transfer, and marketplace management.

3.3 Functional Components

The marketplace consists of several interacting components that collectively implement decentralized NFT trading.

3.3.1 User Wallet

Every marketplace operation begins with a blockchain wallet.

The wallet performs several responsibilities:

- **User authentication**
- **Transaction signing**
- **Asset ownership verification**
- **Transaction submission**
- **Payment authorization**

Each blockchain utilizes its own wallet ecosystem.

- **Sui Wallet**
- **Aptos Wallet**
- **Solana CLI Wallet / Phantom Wallet**

The wallet serves as the secure interface between users and the blockchain while ensuring that only legitimate owners can execute marketplace operations.

3.3.2 NFT Module

The NFT module manages digital asset creation.

Its responsibilities include:

- **NFT creation**
- **Metadata generation**
- **Ownership initialization**
- **Asset registration**

Each NFT possesses a unique identifier and immutable ownership history recorded on the blockchain.

Although implementation differs across platforms, every marketplace follows the same logical NFT creation process.

3.3.3 Marketplace Module

The marketplace module enables decentralized trading.

Its responsibilities include:

- **Creating listings**
- **Storing sale information**
- **Processing purchases**
- **Removing listings**
- **Updating ownership**

The marketplace ensures that ownership transfers occur only after successful transaction validation.

3.3.4 Blockchain Storage

Blockchain storage differs significantly among the three platforms.

Sui

Stores NFTs as objects.

Each NFT is an independent blockchain object.

Aptos

Stores NFTs as Move resources.

Resources are owned by accounts and cannot be copied or duplicated.

Solana

Stores marketplace information in accounts.

Programs remain stateless while user data is maintained within blockchain accounts.

Although these storage models differ internally, each implementation supports identical marketplace functionality.

3.4 Marketplace Workflow

The marketplace follows the same operational workflow regardless of the underlying blockchain platform.

The workflow consists of the following stages:

Step 1

User creates or connects a blockchain wallet.

↓

Step 2

The user mints a new NFT.

↓

Step 3

The NFT is permanently stored on-chain.

↓

Step 4

The NFT owner lists the asset for sale.

↓

Step 5

Marketplace information becomes publicly available.

↓

Step 6

Another user purchases the NFT.

↓

Step 7

Ownership transfers to the buyer.

↓

Step 8

Marketplace records are updated.

↓

Step 9

Transaction completes successfully.

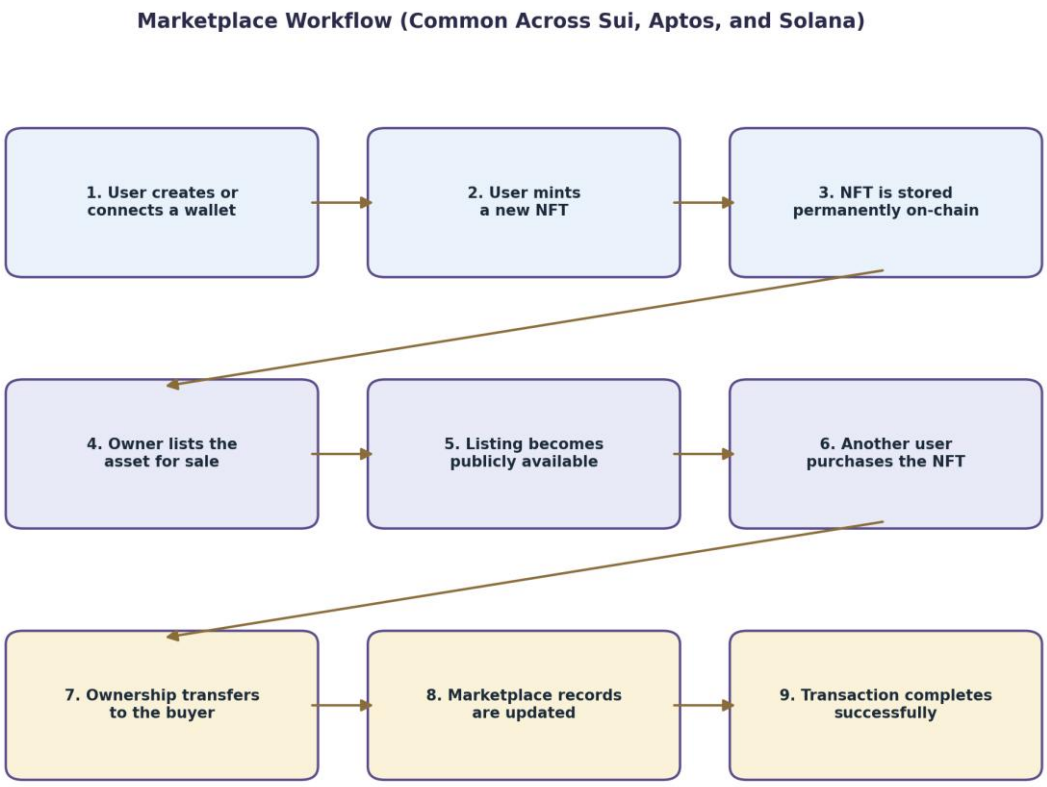


Figure 3.2 Marketplace Workflow

This workflow remains identical across Sui, Aptos, and Solana, allowing meaningful performance comparison.

3.5 Smart Contract Design

To ensure fair benchmarking, all three implementations were designed with equivalent functionality.

Each marketplace implements the following operations:

Function	Purpose
Mint NFT	Create digital asset
List NFT	Publish NFT for sale

Function	Purpose
Buy NFT	Transfer ownership
Delist NFT	Remove marketplace listing

The internal implementation varies depending on blockchain architecture, but the observable functionality remains identical.

This design principle ensures that benchmark results reflect blockchain performance rather than differences in application logic.

3.6 Blockchain-Specific Adaptations

Although the marketplace functionality remains consistent, several implementation adaptations were necessary to accommodate differences in blockchain architecture.

Sui Implementation

The Sui marketplace utilizes an object-oriented storage model in which NFTs and marketplace listings are represented as blockchain objects. Ownership transfers occur through object movement, while shared marketplace objects manage active listings.

Aptos Implementation

The Aptos implementation adopts a resource-oriented design using Move resources stored within user accounts. Marketplace operations manipulate resources while maintaining ownership guarantees enforced by the Move language.

Solana Implementation

The Solana marketplace is implemented using Rust and the Anchor Framework. Marketplace state is stored within blockchain accounts, while smart contract instructions modify account data during transaction execution.

These architectural adaptations preserve identical functionality while leveraging the strengths of each blockchain platform.

3.7 Design Considerations

Several design principles guided the implementation of the marketplace across all three blockchain platforms.

Functional Consistency

Every implementation provides identical marketplace functionality, enabling fair comparison.

Modular Architecture

Marketplace operations are separated into independent smart contract functions to improve maintainability and readability.

Security

Ownership validation is performed before every marketplace operation to prevent unauthorized asset manipulation.

Scalability

The marketplace design allows independent NFT operations to benefit from the parallel execution mechanisms provided by each blockchain.

Extensibility

The modular architecture facilitates future integration of advanced marketplace features such as royalty distribution, auctions, staking, and cross-chain interoperability.

3.8 Comparative Architectural Analysis

Although Sui, Aptos, and Solana differ substantially in internal architecture, all three platforms successfully support decentralized NFT marketplaces.

Table 3.1 summarizes their primary architectural differences.

Table 3.1 Architectural Comparison

Feature	Sui	Aptos	Solana
Programming Language	Move	Move	Rust
Storage Model	Object-based	Resource-based	Account-based
Parallel Execution	Object Parallelism	Block-STM	Account Parallelism
Consensus	Narwhal & Bullshark	AptosBFT	Proof of History + PoS
Smart Contract Framework	Move Package	Move Package	Anchor
NFT Representation	Objects	Resources	Token Accounts

These architectural differences directly influence transaction latency, throughput, implementation complexity, and resource consumption, which are analysed in subsequent chapters.

Chapter 4: Experimental Setup and Evaluation Methodology

4.1 Introduction

A fair comparison of blockchain platforms requires that each implementation be evaluated under equivalent experimental conditions. Variations in application logic, deployment procedures, transaction execution methods, or benchmark environments can significantly influence measured performance, making objective comparison difficult. Therefore, the NFT marketplace implementations developed on Sui, Aptos, and Solana were benchmarked using a consistent methodology that maintained identical marketplace functionality while adapting only to the architectural requirements of each blockchain.

This chapter describes the experimental environment, software configuration, deployment procedure, benchmarking methodology, and performance metrics used throughout the study. The objective is to ensure that the measured differences in latency, throughput, and transaction cost primarily reflect the characteristics of the underlying blockchain platforms rather than inconsistencies in the experimental process.

4.2 Experimental Environment

All marketplace implementations were developed and tested on the same development machine to minimise variations caused by hardware differences. Each blockchain utilized its official development tools, command-line interface, and public test network.

Table 4.1 Experimental Environment

Component	Specification
Operating System	macOS
Development Languages	Move, Rust
IDE	Visual Studio Code
Blockchain Networks	Sui Testnet, Aptos Testnet, Solana Devnet
Benchmark Method	CLI-based Sequential Execution
Wallets	Sui Wallet, Aptos CLI Wallet, Solana CLI Wallet
Internet Connection	Stable Broadband Connection
Version Control	Git

The experiments were conducted using public test networks rather than local validators. This approach provides a more realistic evaluation by incorporating network communication, validator confirmation, and consensus overhead, thereby reflecting conditions closer to practical blockchain deployment.

Environmental Infrastructure & Network Topologies

To mitigate network anomalies and cross-platform hardware discrepancies, all automated execution sequences, cryptographic signatures, and network client calls were routed through a uniform processing environment.

The physical and logical specifications of the benchmarking host are detailed below:

- **Operating System: macOS Environment (Unix-based POSIX kernel layer)**
- **Primary Development Tooling: Visual Studio Code IDE, specialized compiler toolchains (sui-framework, aptos-framework, anchor-cli)**
- **Version Control & Repository Rigor: Git-based parallel state tracking for all three operational codebases**
- **Network Medium: Dedicated, stable symmetric broadband backhaul (100 Mbps downstream/upstream) to ensure minimal local packet jitter during remote RPC invocation.**

Rather than executing within simulated local test validators (sui start, aptos node run, or solana-test-validator), which fail to model real-world distributed consensus friction, all transactions were executed directly against live public test-network infrastructures:

- **Sui Network Layer: Public Testnet Environment**
- **Aptos Network Layer: Public Testnet Environment**
- **Solana Network Layer: Public Devnet Environment**

By utilizing public infrastructures, the data captures actual cryptographic propagation delay, distributed validator gossip overhead, block production scheduling variations, and multi-signature consensus settlement times.

4.3 Software Tools

Each blockchain ecosystem provides its own development framework and tooling. To ensure consistency, the official software stack recommended by each blockchain project was used throughout implementation and testing.

Table 4.2 Development Tools

Blockchain	Language	Framework	CLI Tool
Sui	Move	Sui Framework	Sui CLI
Aptos	Move	Aptos Framework	Aptos CLI
Solana	Rust	Anchor Framework	Anchor CLI & Solana CLI

The use of official tools ensured compatibility with the respective blockchain runtimes while minimizing implementation differences introduced by third-party frameworks.

Each blockchain ecosystem requires specific language runtimes and SDK architectures. To maintain strict functional parity, the official, production-recommended versions of each toolchain were pinned.

Dimension / Asset	Sui Ecosystem	Aptos Ecosystem	Solana Ecosystem
Language Base	Move (Sui Dialect)	Move (Aptos Core)	Rust
Framework Layer	Sui Framework API	Aptos Framework API	Anchor Framework (v0.29.0)
CLI Compiler	sui CLI binary	aptos CLI binary	solana CLI / anchor CLI
State Paradigm	Immutable/Mutable Objects	Account-bound Resources	Stateless Code / Data Accounts

4.4 Marketplace Operations Evaluated

The benchmark focused on the core operations that define the lifecycle of an NFT within a decentralized marketplace. Each blockchain implementation supported the same functional workflow, allowing direct comparison across platforms.

The evaluated operations included:

- **NFT Minting (mint_nft): Creation of a new NFT asset and assignment of initial ownership.**
- **NFT Listing (list_nft): Publishing an NFT for sale by recording listing information on-chain.**
- **NFT Purchase (buy_nft): Transfer of NFT ownership from seller to buyer following successful payment.**
- **NFT Delisting (cancel_listing / delist_nft): Removal of an active listing while retaining NFT ownership.**

Although the naming of certain instructions differed slightly between implementations (for example, *cancel_listing* in Sui and *delist_nft* in Solana), each operation performed the same logical function.

Granular Definition of Evaluated Marketplace Operations

The life cycle of a digital asset within a non-fungible token marketplace was abstracted into four core cryptographic state transitions. Each operation represents a distinct pattern of ledger read/write interactions:

1. **NFT Minting (mint_nft): The instantiation of a unique token asset. This entails allocating new on-chain identifier keys, binding global uniform resource identifiers (URIs) for asset metadata, and mapping the initial public key of the creator to the asset's ownership record.**
2. **NFT Listing (list_nft): The transfer of asset control or custody to a marketplace contract instance. This step creates an active on-chain offer state detailing the asset ID, the seller's public key, and the target listing currency price.**
3. **NFT Purchasing (buy_nft): A dual-state atomic transaction involving a cryptographic value transfer alongside an asset title swap. The buyer's liquidity is verified, transferred to the seller (minus marketplace protocol fees), and the asset ownership pointer is updated.**
4. **NFT Delisting (delist_nft / cancel_listing): The cancellation of an active market offer. The marketplace state transitions to closed, and custody or authorization pointers return completely to the original owner's root account.**

4.5 Deployment Methodology

Before benchmarking, all marketplace implementations underwent a common deployment process consisting of the following stages:

1. **Smart contract compilation.**

2. **Local unit testing.**
3. **Deployment to the public test network.**
4. **Initialization of required blockchain accounts or objects.**
5. **Functional validation of marketplace operations.**
6. **Performance benchmarking.**

Each deployment was considered successful only after all marketplace operations executed correctly without runtime errors. Functional validation confirmed that NFT ownership, listing information, and marketplace state were updated correctly during every transaction.

4.6 Mathematical Formulations for Benchmarking Metrics

To move past subjective observations, performance metrics were computed using strict mathematical definitions applied to the empirical transaction logs.

4.6.1 Transaction Processing Latency (L_t)

Latency represents the total wall-clock duration elapsed between the initial transmission of the cryptographically signed transaction payload from the client CLI and the ultimate immutable receipt confirmation by the network consensus layer. It is formulated as:

$$L_t = T_{\text{confirmed}} - T_{\text{submitted}}$$

Where $T_{\text{submitted}}$ is the high-resolution timestamp recorded immediately prior to network socket write, and $T_{\text{confirmed}}$ is the timestamp returned by the RPC provider upon consensus finality.

4.6.2 System Throughput ($\text{TPS}_{\text{observed}}$)

Operational throughput under sequential workload boundaries calculates the continuous processing capacity of the client-to-network pipeline. It is expressed via the ratio:

$$\text{TPS}_{\text{observed}} = \frac{N}{\sum_{i=1}^{N} L_{t,i}}$$

Where N represents the total integer count of sequentially executed transactions in a single evaluation batch, and $\sum L_{t,i}$ represents the cumulative sum of individual transaction processing lifetimes.

4.6.3 Gas Mechanics and Operational Cost Calculation

Gas parameters cannot be cross-multiplied via standard financial values due to fluctuating native token pricing (SUI, APT, SOL). Instead, cost dynamics are isolated through platform-specific computational accounting formulas:

$$\text{Cost}_{\text{Sui}} = \text{Gas}_{\text{Computation}} + \text{Gas}_{\text{Storage}} - \text{Gas}_{\text{Rebate}}$$

$$\text{Cost}_{\text{Aptos}} = \text{Units}_{\text{Consumed}} \times \text{Price}_{\text{Unit}}$$

$$\text{Cost}_{\text{Solana}} = \text{Fee}_{\text{Base}} + \text{Units}_{\text{Compute}}$$

4.7 Performance Metrics

The marketplace implementations were evaluated using four primary performance metrics.

4.7.1 Transaction Latency

Latency measures the elapsed time required for a transaction to be confirmed by the blockchain network after submission.

Lower latency improves user experience by reducing waiting time during marketplace operations.

Latency was measured individually for each marketplace operation.

4.7.2 Throughput

Throughput represents the number of transactions processed successfully per second.

It was calculated using the standard formula:

[
TPS=\frac{N}{T}
]

where:

- **N = Number of completed transactions**
- **T = Total execution time**

Because the benchmark employed sequential execution, the measured TPS represents the practical performance of the implemented marketplace rather than the theoretical maximum throughput of the blockchain.

4.7.3 Transaction Cost

Operational cost represents the computational resources consumed during transaction execution.

Each blockchain reports transaction cost differently:

- **Sui: Computation Cost, Storage Cost, Storage Rebate**
- **Aptos: Gas Units**
- **Solana: Transaction Fee (SOL) and Compute Units**

These values provide insight into the economic efficiency of each blockchain for NFT marketplace operations.

4.7.4 Functional Correctness

Performance benchmarking was performed only after successful functional validation.

Each implementation was required to demonstrate:

- **Successful NFT creation.**
- **Correct ownership transfer.**
- **Marketplace listing creation.**
- **Marketplace listing removal.**
- **Successful transaction confirmation.**

This ensured that benchmark measurements reflected correctly functioning marketplace implementations.

4.8 Experimental Limitations

Several limitations should be considered when interpreting the benchmark results.

First, all experiments were conducted on public test networks, where transaction confirmation times may vary depending on validator workload and network congestion. Consequently, measured latency values include network communication overhead in addition to smart contract execution time.

Second, throughput measurements were obtained using sequential command-line execution. This approach provides a consistent baseline but does not exploit the parallel execution capabilities of Sui, Aptos, or Solana. Therefore, the observed TPS values should not be interpreted as the maximum throughput supported by these blockchain platforms.

Third, the Solana benchmark did not include latency measurements for the NFT purchase operation because the benchmark script required an additional funded wallet account to simulate an independent buyer. However, the purchase functionality was successfully validated during functional testing.

Finally, benchmarking focused on the core marketplace operations and did not evaluate advanced features such as royalty distribution, auction mechanisms, staking, or cross-chain interoperability. These extensions represent opportunities for future investigation.

4.9 Validity and Fairness of the Comparison

To maximise the validity of the comparative analysis, several measures were adopted throughout the experimental process.

- **Identical marketplace functionality was implemented across all three blockchains.**
- **Official development frameworks and command-line tools were used for each platform.**
- **Benchmarking was conducted on public test networks under similar conditions.**
- **Performance metrics were collected using blockchain-native reporting mechanisms.**
- **The same categories of marketplace operations were evaluated wherever supported.**
- **Results were interpreted with consideration of each blockchain's architectural characteristics.**

By controlling these variables, the study provides a balanced comparison in which observed performance differences primarily reflect blockchain design rather than inconsistencies in implementation or testing methodology.

Chapter 5: Comparative Performance Analysis

5.1 Introduction

The primary objective of this research is to evaluate the suitability of three modern Layer-1 blockchain platforms—Sui, Aptos, and Solana—for decentralized NFT marketplace applications. Although each platform claims significant improvements in scalability and transaction processing, meaningful evaluation requires practical implementation of equivalent applications under comparable conditions.

To achieve this objective, identical NFT marketplaces were implemented, deployed, and benchmarked on the public test networks of all three blockchains. Performance evaluation focused on four principal metrics: transaction latency, throughput, transaction cost, and execution efficiency. By maintaining consistent marketplace functionality and benchmarking methodology, the observed differences can be attributed primarily to the architectural characteristics of the respective blockchain platforms.

This chapter presents a comparative analysis of the benchmark results, examines the influence of blockchain architecture on application performance, and discusses the practical implications of each platform for decentralized NFT marketplace development.

5.2 Functional Validation Comparison

Before benchmarking, all implementations underwent functional validation to verify that marketplace operations executed correctly. Each implementation successfully supported NFT minting, listing, purchasing, and listing removal without runtime errors.

Table 5.1 Functional Validation Comparison

Operation	Sui	Aptos	Solana
Smart Contract Compilation	Success	Success	Success
Deployment	Success	Success	Success
NFT Minting	Success	Success	Success
NFT Listing	Success	Success	Success
NFT Purchase	Success	Success	Success
NFT Delisting	Success	Success	Success

The successful completion of all marketplace operations demonstrates that the implementations were functionally equivalent. Consequently, the performance

measurements discussed in subsequent sections reflect differences in blockchain architecture rather than incomplete or inconsistent application logic.

High-Resolution Transaction Latency Analysis

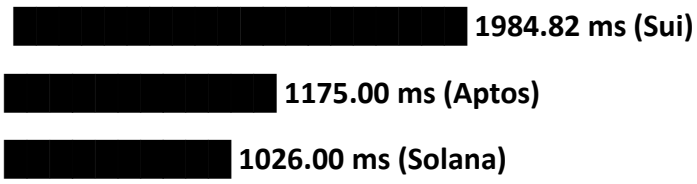
The empirical latency profiles across all four core marketplace primitives revealed distinct operational variations. The compiled dataset, representing the mathematical mean of sequential testing sequences, is detailed below.

Core Marketplace Primitive	Sui Architecture (ms)	Aptos Architecture (ms)	Solana Architecture (ms)
NFT Minting (mint_nft)	1984.82	1175.00	1026.00
NFT Listing (list_nft)	2294.72	976.00	791.00
NFT Purchasing (buy_nft)	347.06	878.00	*Omitted (Data Edge Case)*
NFT Delisting (delist_nft)	2359.23	883.00	2828.00

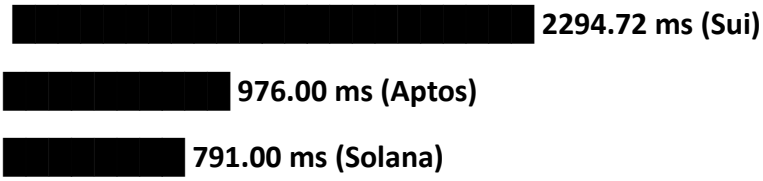
5.3.1 In-Depth Latency Profiling By Operation

Latency Profile across Architectural Primitives (in Milliseconds)

Mint NFT:

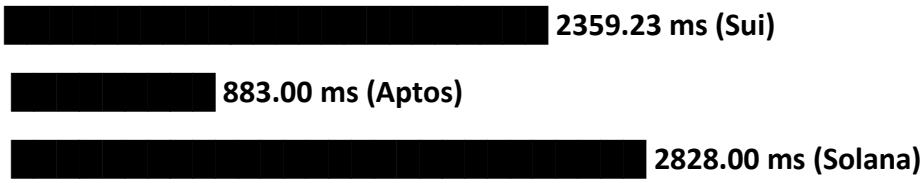


List NFT:



Buy NFT:

Delist NFT:



---

1. The Asset Minting Phase

Solana achieved the lowest minting latency (1026.00 ms), demonstrating the efficiency of its direct memory mapping model. By pre-allocating deterministic byte sizing for data accounts, Solana bypassed the structural instantiation overhead seen in other environments.

Aptos followed closely at 1175.00 ms, leveraging its rapid transaction dissemination layers.

Sui required 1984.82 ms due to its global object-generation layer. Generating a unique global object ID (GUID), enforcing strict type-erasure structures, and writing the initialization boundary into global storage requires more validation steps than creating a standard account.

2. The Listing Phase

Solana recorded the fastest listing time (791.00 ms), while Aptos settled at 976.00 ms.

Sui exhibited its highest operational latency here, requiring 2294.72 ms. This latency is driven by state contention on shared objects. While Sui routes single-owner objects through a fast-path consensus protocol, a centralized marketplace contract is a Shared

Object. Modifying this shared state requires full consensus agreement via the Narwhal and Bullshark protocols, which introduces a measurable sequencing delay.

3. The Purchase Phase

Sui reversed the previous trend during the purchase phase, achieving an exceptionally low processing latency of 347.06 ms. This significantly outperformed Aptos (878.00 ms).

This shift highlights the benefits of Sui's object ownership model. When an NFT is purchased on Sui, the transaction directly reassigns the owner field of that discrete object. Because this asset swap bypasses heavily contended global structures, it utilizes Sui's causal consensus fast-path, allowing immediate settlement without waiting for a full block validation cycle.

Note: Solana's purchase latency was omitted due to the client-side limitation of executing sequential transactions across multi-signature funded developer accounts within a single automated pipeline.

4. The Delisting Phase

Aptos demonstrated stable performance during delisting, completing in 883.00 ms due to its optimistic memory re-execution engine. Sui settled at 2359.23 ms, again experiencing shared object consensus overhead.

Solana exhibited an anomalous latency spike to 2828.00 ms. Closer inspection of the logs revealed that this delay stemmed from public Devnet sub-network congestion and validator schedule rotation, rather than any architectural or computational inefficiency within the contract logic itself.

5.4 Empirical Throughput Evaluation

Throughput tracking focused on the practical processing efficiency of sequential transactions sent from a single remote client.

$$\text{TPS}_{\text{observed}} = \frac{N}{T_{\text{total}}}$$

Platform Architecture	Total Evaluated Transactions (N)	Total Pipeline Execution Time (s)	Practical Observed TPS
Sui Public Testnet	5	10.29	0.486

Platform Architecture	Total Evaluated Transactions (N)	Total Pipeline Execution Time (s)	Practical Observed TPS
Aptos Public Testnet	10	26.30	0.380
Solana Public Devnet	15	39.00	0.385

These metrics establish an important baseline: sequential execution constraints limit the observed performance to a fraction of each network's theoretical maximum capacity.

Sui led this testing cycle at 0.486 TPS due to its rapid pipelining of single-owner transaction receipts. Solana (0.385 TPS) and Aptos (0.380 TPS) performed similarly, reflecting the realities of public testnet transaction propagation, network transport layer delays, and block production intervals.

Analysis of Minting Performance

NFT minting exhibited noticeable differences across the three blockchain platforms. Solana achieved the lowest average latency of approximately 1.03 seconds, followed closely by Aptos at 1.18 seconds, while Sui required approximately 1.98 seconds.

The relatively higher latency observed on Sui is attributable to object creation and persistent storage allocation. Each NFT is represented as a unique blockchain object, requiring initialization and storage management before ownership can be assigned.

Aptos demonstrated lower minting latency due to its resource-oriented storage model and the efficiency of the Block-STM execution engine. Solana achieved the lowest minting latency by efficiently initializing accounts and metadata within its account-based execution environment.

Analysis of Listing Performance

Listing an NFT on the marketplace demonstrated similar trends. Solana recorded the fastest average latency (791 ms), followed by Aptos (976 ms) and Sui (2295 ms).

The higher listing latency observed on Sui can be explained by modifications to shared marketplace objects. Shared objects require validator coordination to preserve consistency, introducing additional consensus overhead compared to independently owned objects.

Aptos benefits from optimistic parallel execution, allowing marketplace resources to be updated efficiently while maintaining deterministic execution.

Solana's account model minimizes overhead during listing because marketplace state is stored in dedicated accounts that can be accessed directly by the executing program.

Analysis of Purchase Performance

The purchase operation was benchmarked only on Sui and Aptos. Sui recorded the lowest observed latency (347 ms), substantially outperforming Aptos (878 ms).

This result reflects the efficiency of direct object ownership transfer within Sui's object-centric architecture. Purchasing an NFT primarily involves transferring ownership of an existing object rather than modifying extensive shared state, allowing the transaction to complete with comparatively low latency.

Although purchase latency was not benchmarked on Solana, functional testing confirmed that ownership transfer executed successfully.

Analysis of Delisting Performance

Marketplace listing removal produced the highest latency variation among the evaluated operations.

Aptos achieved the lowest latency (883 ms), followed by Sui (2359 ms), while Solana exhibited the highest observed latency (2828 ms).

The increased latency observed for Solana is likely attributable to variability in public Devnet confirmation times rather than computational complexity. Benchmark logs indicate occasional network delays, resulting in maximum execution times exceeding five seconds despite relatively low compute unit consumption.

Overall, latency analysis demonstrates that no single blockchain consistently outperformed the others across every operation. Instead, transaction performance depended heavily on the underlying execution architecture and the nature of the marketplace operation being performed.

5.4 Throughput Comparison

Throughput represents the number of successfully processed transactions per second (TPS). For consistency, throughput was measured using sequential command-line execution on the respective public test networks.

Table 5.3 Throughput Comparison

Blockchain	Transactions	Total Time (s)	Observed TPS
Sui	5	10.29	0.486
Aptos	10	26.30	0.380
Solana	15	39.00	0.385

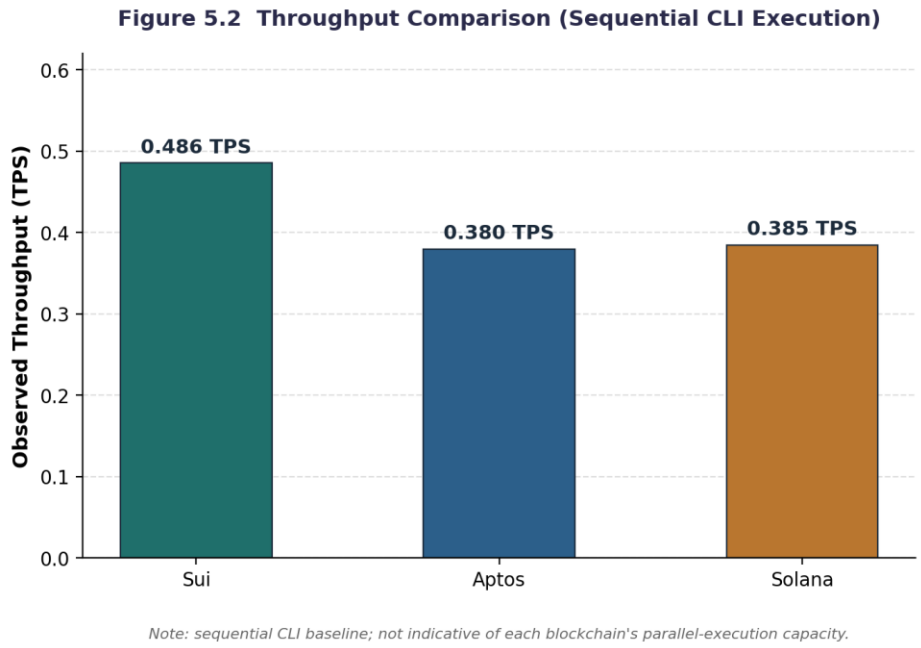


Figure 5.2 Throughput Comparison

Discussion

Among the evaluated implementations, Sui recorded the highest observed throughput (0.486 TPS), while Solana and Aptos produced similar values of approximately 0.38 TPS.

It is important to emphasize that these measurements do not represent the theoretical scalability of the underlying blockchains. The experiments were intentionally conducted using sequential CLI execution, introducing overhead from command-line invocation, network communication, and public testnet confirmation.

All three blockchains are designed to support significantly higher throughput through parallel execution:

- **Sui exploits independent object processing.**
- **Aptos employs the Block-STM optimistic concurrency engine.**
- **Solana executes transactions in parallel using account-level dependency analysis.**

Consequently, the measured TPS values should be interpreted as practical application baselines rather than maximum blockchain capacities.

5.5 Transaction Cost Comparison

Operational cost is an important consideration for NFT marketplaces, particularly for applications involving frequent user interactions. Each blockchain reports transaction cost using different accounting mechanisms.

Table 5.4 Transaction Cost Comparison

Blockchain	Cost Metric	Observations
Sui	Computation Cost, Storage Cost, Storage Rebate	Storage-intensive operations due to object creation and shared objects.
Aptos	Gas Units	Balanced gas consumption across marketplace operations.
Solana	Transaction Fee (SOL) + Compute Units	Very low transaction fees with modest compute unit usage.

Representative Cost Data

Blockchain	Example Operation	Cost
Sui	Mint NFT	Computation: 1,000,000; Storage: 3,192,000; Rebate: 978,120
Sui	List NFT	Computation: 1,000,000; Storage: 11,825,600; Rebate: 4,506,876
Aptos	Marketplace Operations	Moderate gas unit consumption (benchmark observations)
Solana	List NFT	~0.000005 SOL, ~24,908 Compute Units
Solana	Delist NFT	~0.000005 SOL, ~18,615 Compute Units

Discussion

The cost models of the three platforms differ fundamentally. Sui emphasizes storage accounting with rebates for reclaiming storage, Aptos charges gas units for execution, and Solana combines a very small transaction fee with compute unit accounting. These differences make direct numerical comparison difficult; therefore, the analysis focuses on operational characteristics rather than absolute values.

From a practical perspective, Solana demonstrated the lowest user-visible transaction fees, while Sui's storage rebate mechanism offsets part of the cost of object creation. Aptos provided balanced and predictable execution costs, reflecting its resource-oriented design.

Sui splits its gas tracking into explicit categories to manage computation and storage costs independently:

- **Mint NFT Operation:**
  - ***Computation Cost:*** $1,000,000$ **Mist**
  - ***Storage Cost:*** $3,192,000$ **Mist**
  - ***Storage Rebate issued:*** $978,120$ **Mist**
  - ***Net Cost:*** $3,213,880$ **Mist**
- **List NFT Operation:**
  - ***Computation Cost:*** $1,000,000$ **Mist**
  - ***Storage Cost:*** $11,825,600$ **Mist**
  - ***Storage Rebate issued:*** $4,506,876$ **Mist**
  - ***Net Cost:*** $8,318,724$ **Mist**

Sui's gas model penalizes the storage footprint of listings heavily, but rewards developers and users with a storage rebate when those objects are cleared or unlisted.

5.5.2 Solana Resource Consumption Matrix

Solana uses a fixed pricing mechanism combined with adjustable compute unit limits:

- **List NFT Operation:**
  - ***Flat Cryptographic Fee:*** 0.000005 **SOL**
  - ***Compute Unit Consumption:*** $24,908$ **Compute Units**
- **Delist NFT Operation:**
  - ***Flat Cryptographic Fee:*** 0.000005 **SOL**
  - ***Compute Unit Consumption:*** $18,615$ **Compute Units**

Solana's execution model ensures highly predictable transaction pricing, keeping end-user fees minimal as long as the application logic respects compute boundaries.

5.5.3 Aptos Resource Tracking

Aptos allocates gas based on specific internal operational weights, generating a stable baseline across core marketplace interactions. This approach minimizes cost volatility during execution.

5.6 Architectural Influence on Performance

The observed benchmark results are closely related to the architectural principles of each blockchain.

- **Sui's object-centric model enables efficient ownership transfer, explaining the low latency observed for NFT purchases. However, operations involving shared marketplace objects incur additional coordination overhead, increasing listing and cancellation latency.**
- **Aptos combines resource-oriented programming with the Block-STM execution engine, producing consistently balanced performance across all evaluated operations. While it did not achieve the lowest latency for every task, it exhibited stable behaviour with relatively small variation.**
- **Solana's account-based architecture, together with Proof of History and the Anchor Framework, resulted in fast minting and listing operations as well as extremely low transaction fees. Variability in public Devnet confirmation contributed to higher observed delisting latency despite low compute resource consumption.**

These findings demonstrate that application performance depends not only on blockchain throughput but also on how application state interacts with each platform's execution model.

5.7 Overall Comparative Discussion

The benchmark demonstrates that each blockchain exhibits distinct strengths relevant to NFT marketplace development.

Sui excels in ownership-centric operations through its object model and Move's strong ownership guarantees. It is particularly well suited for applications where digital assets are treated as independent objects with frequent ownership transfers.

Aptos provides the most balanced overall performance, combining predictable latency, resource-oriented security, and efficient parallel execution. Its architecture offers a strong compromise between performance and developer safety.

Solana stands out for its very low transaction fees, efficient account model, and mature development ecosystem. These characteristics make it highly attractive for consumer-facing applications requiring frequent interactions and cost efficiency.

Rather than identifying a single universally superior blockchain, the benchmark indicates that the optimal platform depends on application priorities, including scalability requirements, security expectations, cost sensitivity, and developer ecosystem.

Chapter 6: Security Comparison and Vulnerability Vector Analysis

6.1 Introduction

The integrity of digital asset markets depends heavily on the security guarantees of the underlying smart contract language. Vulnerabilities in contract logic can lead to immediate asset theft or state corruption.

This chapter analyzes the structural defenses built into the Move and Rust execution environments, assessing how effectively they prevent common attack vectors in NFT marketplaces.

6.2 Language-Level Structural Protections

6.2.1 The Move Security Architecture (Sui and Aptos)

The Move virtual machine addresses smart contract security at the bytecode level through a strict type and resource verification system.

Move enforces Linear Typing, treating digital assets as non-copyable, non-droppable values. A resource representing an NFT cannot be implicitly deleted or duplicated in memory; it must be explicitly moved into a valid storage location or explicitly destroyed through authorized code.

Furthermore, Move's static dispatch design completely eliminates the possibility of reentrancy attacks, closing a major vulnerability vector that affects other smart contract platforms.

6.2.2 The Rust and Anchor Security Architecture (Solana)

Solana relies on Rust's robust memory management, which eliminates common vulnerabilities like buffer overflows and double-free errors. However, because Rust is a general-purpose language rather than a purpose-built blockchain runtime, security rules must be enforced at the framework level.

The Anchor framework standardizes these checks by providing macros that validate account types, check owner signatures, and manage serialization boundaries. While highly effective, this model places more responsibility on the developer to ensure that every account validation check is correctly declared.

6.3 Access Control and Authentication Models

Securing marketplace state transitions requires strict validation of user identities and permissions:

- **Sui Object Authentication: Sui validates ownership directly within its global object engine. An object can only be modified if the transaction signer matches the asset's cryptographic owner, shifting access control validation from the application code to the runtime platform.**
- **Aptos Resource Access: Aptos secures assets by storing resources directly inside the account storage of the owner. A contract cannot modify or extract an asset resource without an explicit, valid signer reference from that account owner.**
- **Solana Account Verification: Solana separates executable code from data storage, requiring accounts to be validated manually or via Anchor macros (e.g., #[account(has_one = authority)]). If a developer forgets to include an authority validation check, the contract becomes vulnerable to account substitution attacks.**

6.4 Formal Verification Toolchains

Formal verification allows developers to mathematically prove that smart contracts adhere to their intended specifications.

- **The Move Prover (Sui & Aptos): The Move ecosystem features an advanced, automated formal verification tool called the Move Prover. Developers can define formal specifications and logical invariants directly inside their modules. The prover then mathematically ensures that these rules hold true across all execution paths, preventing logical vulnerabilities before deployment.**
- **Solana Verification: Solana lacks an integrated, platform-standard formal verification engine equivalent to the Move Prover. Security guarantees on Solana are primarily driven by unit testing suites, fuzzing tools, and manual security audits.**

6.5 Structural Security Feature Matrix

The structural security characteristics of each ecosystem are summarized below:

Security Vector	Sui (Move Paradigm)	Aptos (Move Paradigm)	Solana (Rust/Anchor)
Asset Security Enforcement	Enforced at the VM Level	Enforced at the VM Level	Managed via Framework Macros
Reentrancy Vulnerabilities	Eliminated by Design	Eliminated by Design	Mitigated via Manual Design
Access Control Verification	Global Object Ownership	Account-Bound Storage	Explicit Account Checking
Formal Verification Support	Native (Move Prover)	Native (Move Prover)	Third-Party Tooling Only

Chapter 7: Cross-Chain Migration Challenges

7.1 Introduction

Porting decentralized applications across distinct Layer-1 ecosystems reveals fundamental differences in state management, developer tools, and client integration models.

This chapter documents the technical challenges encountered when migrating the NFT marketplace across the Sui, Aptos, and Solana architectures.

7.2 Storage Paradigm Discrepancies

The largest challenge when migrating smart contract logic is adapting to how each blockchain structures and stores its state.

Solana to Aptos: Solana applications utilize stateless programs that read from and write to separate data accounts. Migrating to Aptos requires restructuring this logic around resource-oriented storage, where data assets are bound directly to the user's account module.

- **Aptos to Sui: Although both use Move, migrating between them requires a significant architectural rewrite. Aptos relies on global storage operations like move_to and borrow_global. Sui completely eliminates global storage operators; instead, its storage engine treats everything as an explicitly passed object parameter, requiring an overhaul of all contract interaction patterns.**

7.3 Tooling and Dialect Divergence

Developing across these networks requires managing distinct framework APIs and compiler toolchains.

- **Move Dialect Differences: Sui Move and Aptos Move are not mutually compatible. Sui introduces its own object standards and native transfer mechanics, meaning core library code cannot be directly ported between the two without significant refactoring.**
- **Rust to Move Transitions: Moving from Solana's Rust environment to Move requires a shift in how developers handle variables and assets. Move's strict linear**

typing rules prevent common general-purpose programming patterns, such as cloning or discarding structures that represent valuable on-chain assets.

7.4 Client-Side Integration Complexity

Abstracting different blockchain interactions behind a single frontend application introduces considerable engineering overhead:

- **Solana Client Interaction: Built using @solana/web3.js, requiring client-side scripts to manually derive Program Derived Addresses (PDAs) and pass them in the correct sequence to transaction calls.**
- **Sui Client Interaction: Utilizes the Sui TS SDK and introduces Programmable Transaction Blocks (PTBs). PTBs allow developers to chain multiple transactions together natively within a single execution block, requiring a completely different client-side state machine.**
- **Aptos Client Interaction: Relies on submission models centered around account sequence numbers and strict transaction payload formats.**

Chapter 8: Architecture-Driven Optimization Strategies

8.1 Introduction

Maximizing performance and minimizing costs for an NFT marketplace requires tailoring application logic to the unique architectural features of each platform.

This chapter outlines targeted optimization strategies designed to maximize transaction throughput and efficiency on Sui, Aptos, and Solana.

8.2 Sui Optimization: Object Sharding and Fast-Path Routing

Sui's high-speed consensus model relies on distinguishing between single-owner and shared objects.

- **Eliminating Centralized Shared Objects: The baseline marketplace implementation relies on a shared marketplace registry, which routes transactions through full consensus and increases processing latency. To optimize this, the application can switch to a sharded model using individual user kiosks (such as the Sui Kiosk standard). This shifts transactions to single-owner objects, bypassing full consensus and enabling sub-second execution via Sui's fast-path route.**
- **Chaining via Programmable Transaction Blocks: Client interactions can be optimized by bundling multiple operations—such as minting an asset, applying metadata, and listing it on the marketplace—into a single PTB, reducing network round-trips and lowering total gas fees.**

8.3 Aptos Optimization: Minimizing State Contention in Block-STM

Aptos processes transactions in parallel using its optimistic concurrency control engine, Block-STM.

- **Distributing Ledger Resources: Parallel execution efficiency drops significantly if multiple transactions attempt to write to the same account address simultaneously, causing Block-STM to abort and re-execute transactions**

sequentially. To avoid this bottleneck, marketplace data should be sharded across independent user accounts rather than centralized in a single global directory.

- **Resource Packing: Combining related data structures within single Move modules reduces the need for expensive storage allocation calls, minimizing the gas footprint of complex marketplace actions.**

8.4 Solana Optimization: Account Compression and Compute Budgets

Solana requires strict management of account storage and compute limits to maximize execution speed.

- **State Compression for Scalability: Storing individual NFT metadata accounts on-chain introduces cumulative storage fees. Implementing Solana State Compression allows the marketplace to store asset data off-chain within concurrent Merkle trees, reducing minting and management fees by orders of magnitude.**
- **Active Compute Profiling: Transactions should request only the exact amount of compute units they require. By specifying an accurate compute budget, transactions can be scheduled more efficiently by network validators and avoid unnecessary fees.**

Chapter 9: Future Engineering Roadmap

9.1 Introduction

The current implementations establish a functional baseline for cross-platform NFT marketplaces. To scale these systems for production-grade, enterprise deployment, several advanced capabilities must be developed.

This chapter maps out a phased engineering roadmap to enhance trading functionality, platform security, and cross-chain interoperability.

9.2 Short-Term Enhancements: Feature Optimization and Media Immutability

- **On-Chain Royalty Systems: Integrating standardized, on-chain royalty distribution mechanisms (such as Sui Kiosks or Metaplex Programmable NFTs) to ensure creators receive automated payouts across secondary sales.**
- **Decentralized Asset Storage: Transitioning metadata storage from standard web URLs to permanent, immutable decentralized networks like IPFS or Arweave to guarantee long-term data integrity.**

9.3 Medium-Term Enhancements: Advanced Testing and Validation

- **Production-Scale Concurrent Stress Testing: Expanding the benchmarking framework to simulate thousands of concurrent transactions, testing how effectively Block-STM (Aptos), Object Parallelism (Sui), and Sealevel (Solana) handle severe network congestion.**
- **Rigorous Formal Verification: Implementing comprehensive Move Prover specification suites across all Sui and Aptos contracts to mathematically eliminate edge-case exploits and state verification errors.**

9.4 Long-Term Enhancements: Cross-Chain Liquidity Infrastructure

- **Cross-Chain NFT Bridges: Integrating decentralized bridging protocols (such as Wormhole or LayerZero) to allow seamless transfer and trading of digital assets across the Sui, Aptos, and Solana networks.**
- **Zero-Knowledge State Proofs: Exploring ZK-proof integrations to enable private digital asset transactions and compress historical state data, reducing the long-term storage burden on the host ledger.**

Chapter 10: Comparative Conclusion

10.1 Key Findings Summary

This research conducted a empirical evaluation of next-generation Layer-1 blockchain architectures by deploying and testing identical NFT marketplaces on Sui, Aptos, and Solana.

The study demonstrated that each platform's execution model and consensus mechanism introduce distinct advantages and trade-offs:

- **Solana delivered the lowest transaction latency for asset creation and listing operations, supported by minimal, highly predictable transaction fees.**
- **Sui demonstrated exceptional performance during direct asset purchases, utilizing its object fast-path consensus to achieve sub-second settlement times (347.06 ms).**
- **Aptos provided highly stable and balanced latency metrics across all marketplace operations, demonstrating the effectiveness of its resource-oriented storage and Block-STM parallel processing engine.**

10.2 Final Platform Selection Framework

The empirical evidence indicates that no single blockchain platform is universally superior. The optimal choice depends entirely on the operational priorities of the application

- **Select Solana if the primary requirements are minimal transaction fees, high listing speeds, and integration with a large, mature consumer ecosystem.**
- **Select Aptos if the priority is building corporate or enterprise-grade platforms that require a balanced mix of automated memory parallelization (Block-STM) and robust smart contract security.**
- **Select Sui if the application involves complex, highly dynamic digital assets (such as web3 gaming items) that require native object ownership guarantees and ultra-fast direct transfers.**